Statistical and Computational Models

Lecture 19: Deep learning

Daniel de Kadt

Department of Government, Cornell University · Fall 2026

Roadmap

Training big networks

Overfitting

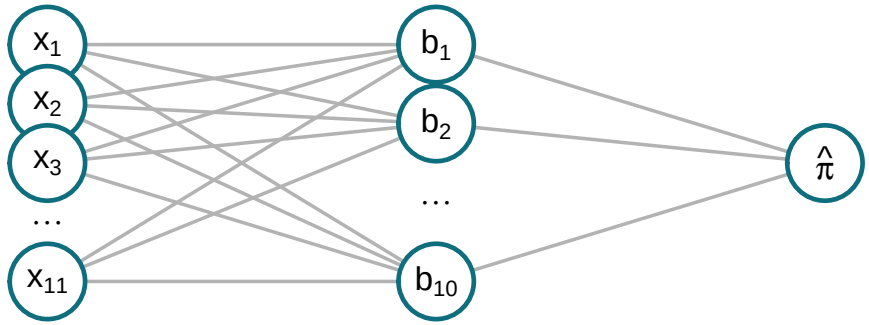
Going deep

Reading tally sheets

Our network so far

Recall our problem: predicting $Y = 1$ (election irregularity). We estimate $\pi(\boldsymbol{x}) = \mathbb{P}(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$.

Last time, we trained this network on our six features ($p = 11$ columns, $J = 10$):



$$\underbrace{(p+1)\,J}_{\boldsymbol{w}\text{s, into the } \boldsymbol{b}_j\text{: } 12\times 10} \quad + \quad \underbrace{J+1}_{\beta\text{s, into } \hat{\pi}\text{: } 10+1} \quad = \quad 131 \text{ weights, together } \boldsymbol{\theta}$$

A **weight** is just a coefficient... But every edge on the network is weighted, not just the final layer. Plus an intercept (aka bias) for each $\boldsymbol{b}_j$ and $\hat{\pi}$.

We chose J and the weight decay, $\lambda = 1$, by CV on the spline's folds.

Training is minimizing

To train our network, we want the weights with the lowest log loss:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \text{ log loss}(\boldsymbol{\theta})$$

Model	Minimizes	How we find the minimum
OLS	RSS	a formula: $\hat{\boldsymbol{\beta}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$
Logit	log loss	Newton–Raphson updates
Our network	log loss	quasi-Newton updates, from ten random starts

Why not try every value? For 131 weights, even 10 values each is 10^{131} fits.

So we do what Newton–Raphson did for our maximum likelihood estimates: start somewhere, update the weights, and repeat. The slope tells us which direction to move, and the curvature tells us how big each update should be.

Training our network

`nnet` trained these 131 weights by quasi-Newton updates:

$$\underbrace{\boldsymbol{\theta}^{(t+1)}}_{\substack{\text{all 131}\\ \text{weights}}} = \underbrace{\boldsymbol{\theta}^{(t)}}_{\substack{\text{all 131}\\ \text{weights}}} - \underbrace{H\left(\boldsymbol{\theta}^{(t)}\right)^{-1}}_{\substack{\text{curvature: } 131 \times 131,\\ \text{one number per pair of weights:}\\ \text{how one's slope shifts}\\ \text{as the other moves}}} \underbrace{s\left(\boldsymbol{\theta}^{(t)}\right)}_{\substack{\text{slope: 131 numbers,}\\ \text{one per weight}}}$$

Network	Weights	Numbers in H
Last time ($J = 10$)	131	17,161
Today's wide network ($J = 256$)	3,329	11 million
ISL's digit network (two layers)	235,146	55 billion

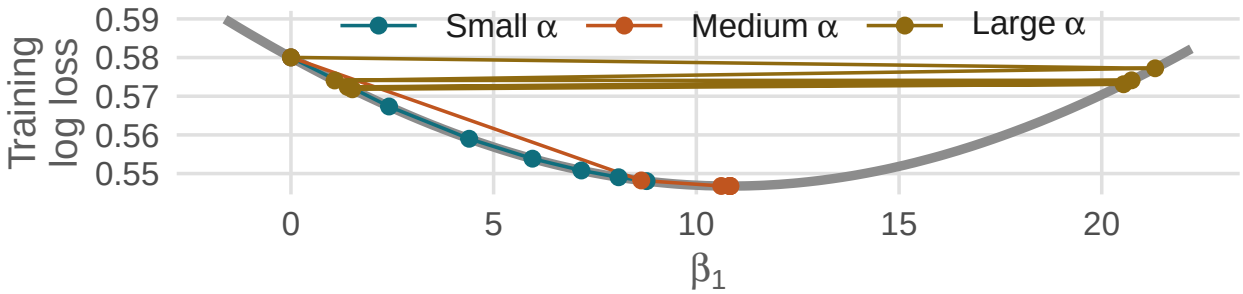
Each update moves all 131 weights at once, and every update needs all of H.

Complex models with many features may have millions of weights. Perhaps we can train them without H.

Gradient descent

Keep the slope, which tells us which way is downhill, but drop the curvature:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \underbrace{\alpha}_{\text{step size}} \underbrace{\frac{\partial \log \text{loss}}{\partial \boldsymbol{\theta}}}_{\text{gradient: one slope per weight}}$$

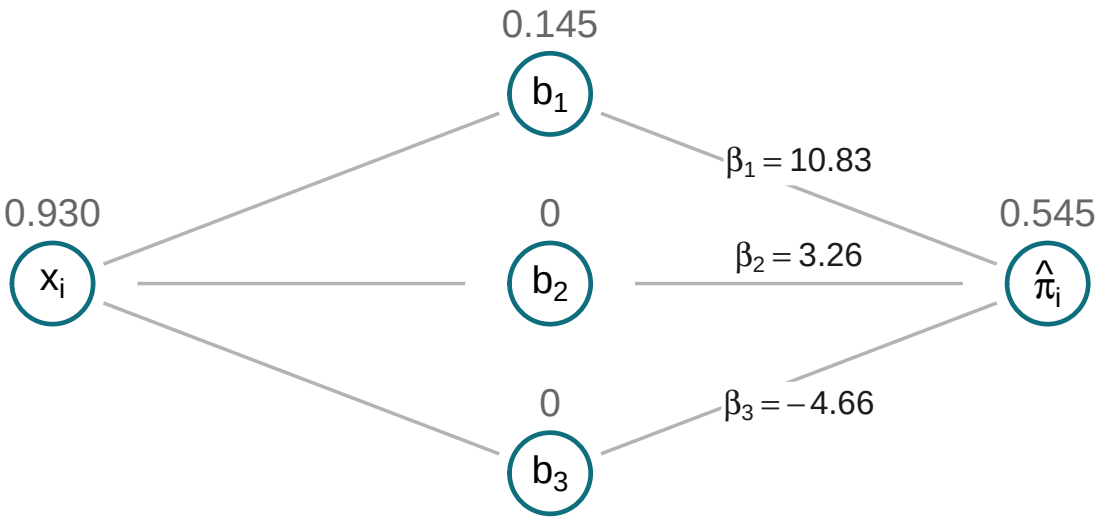


Our turnout network's log loss along β_1, others fixed. From a start of $\beta_1 = 0$, a small α creeps along, while a large α bounces around. A medium α is just right.

This is **gradient descent**. Without the curvature, we must select α (aka the learning rate): a **tuning parameter**, like boosting's step size.

The backward pass

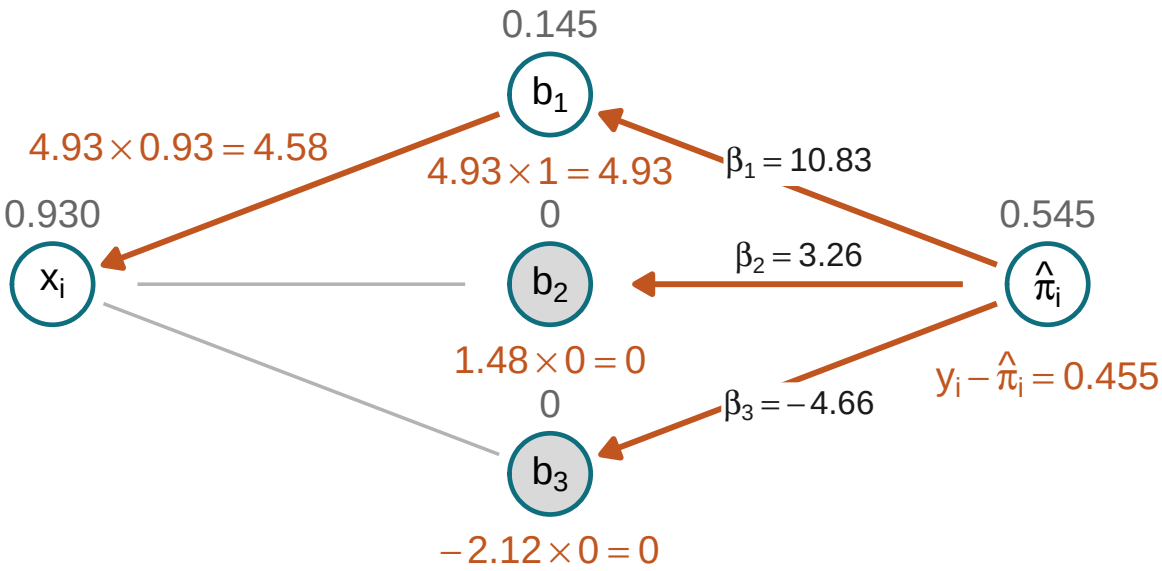
Each update needs the slope of the log loss for every weight. Consider one station with an irregularity ($y_i = 1$), and its log-likelihood, ℓ_i:



The forward pass runs left to right: every node gets its value.

The **backward pass** runs right to left. By the **chain rule**, a weight's slope is the product of the slopes along its path to ℓ_i.

Backpropagation



Start at the output: ℓ_i's slope in the logit is the residual. Move back along each edge by multiplying by β_j. Through each ReLU: times 1 if on, 0 if off. Multiply by x_i back to the input.

After the ReLU, the number we have is the slope for w_{j0}, the intercept of b_j. Multiplying it by x_i gives the slope for w_{j1}. This is **backpropagation**: one residual, reused along every edge, for about the cost of the forward pass.

Estimating the gradient

One backward pass gives us one station's slopes. In the previous example, we got $\partial \ell_i / \partial w_{11} = 4.58$: raise w_{11} a little, and ℓ_i rises 4.58 times as much.

The gradient averages these slopes over all n stations. So each gradient descent update takes a backward pass through every station: one **epoch**.

$$\frac{\partial \log \text{loss}}{\partial \boldsymbol{\theta}} = \underbrace{-\frac{1}{n}\sum_{i=1}^{n} \frac{\partial \ell_i}{\partial \boldsymbol{\theta}}}_{\text{all } n \text{ stations}} \approx \underbrace{-\frac{1}{n_b}\sum_{i \in \text{sample}} \frac{\partial \ell_i}{\partial \boldsymbol{\theta}}}_{\text{a random sample of } n_b}$$

But this is a mean, and we can estimate a mean from a random sample.

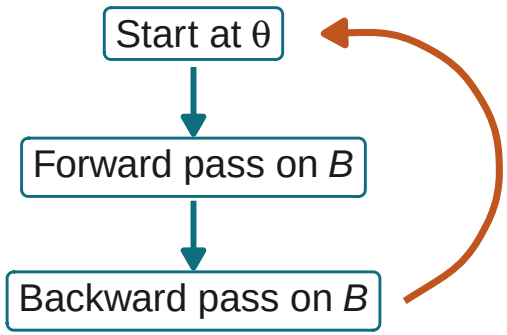
Say $n_b = 32$ of our 4,000 stations, a typical size. The full mean costs 125 times as much, for a standard error only $\sqrt{125} \approx 11$ times smaller.

Would you rather make one precise update, or 125 noisy ones?

Stochastic gradient descent

Instead, each update uses only a small random set of stations:

1. Start from random weights, $\boldsymbol{\theta}$.
2. Repeat for a set number of epochs. In each epoch:
   a. Shuffle the stations, and split them into **minibatches** of n_b (say 32).
   b. Run this loop over the minibatches, using each one once:



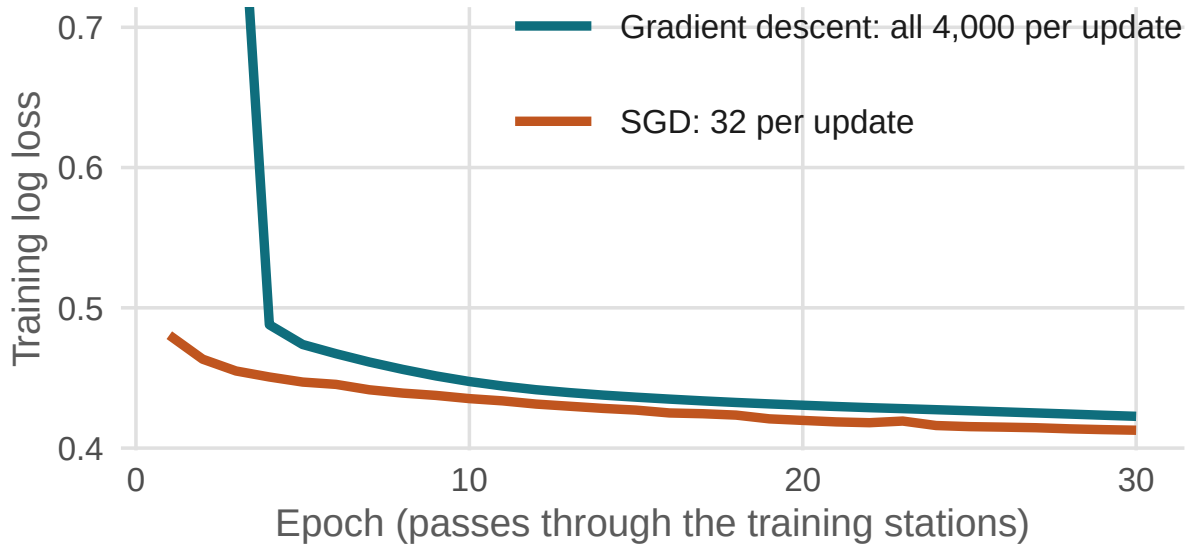
$$\text{Update } \theta \leftarrow \theta - \alpha \frac{\partial \log \text{loss}_B}{\partial \theta}$$

and start on the next minibatch

So in one epoch, SGD gets $n/n_b = 125$ reads of the slope, and makes an update after each. Gradient descent gets one. The cost is that each of the 125 reads is noisy.

SGD on our stations

Is a noisier update worth it?



One network (J = 16, 209 weights), trained for 30 epochs. Gradient descent makes 30 updates with its best α, 4. SGD makes 3,750 with α = 0.05.

After 30 epochs, gradient descent reaches a training log loss of 0.423. SGD reaches the same training log loss after 19 epochs.

Note: for small models, `nnet`'s quasi-Newton updates are still the right tool.

Roadmap

Training big networks

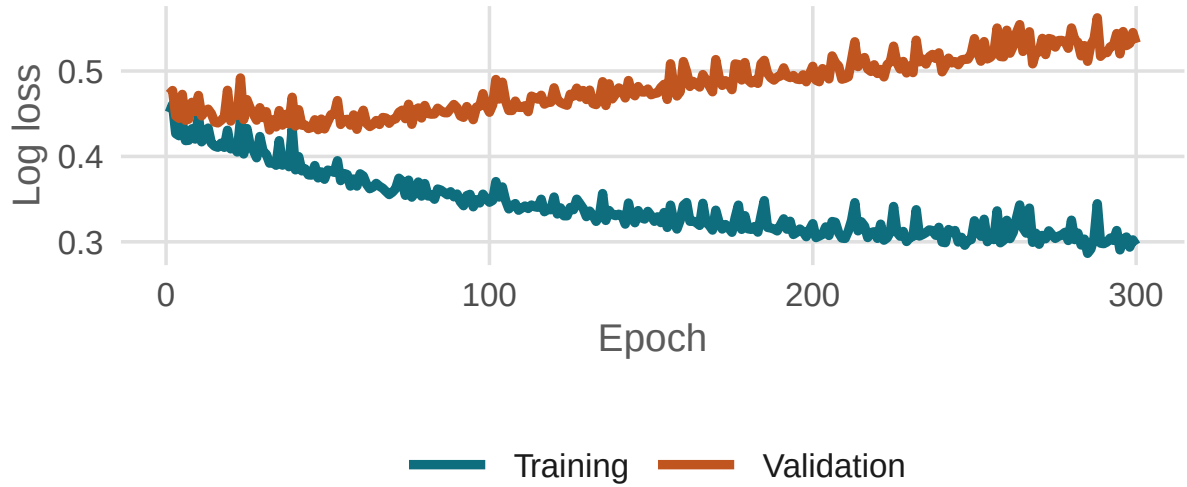
Overfitting

Going deep

Reading tally sheets

Training curves

A wide network ($J = 256$, 3,329 weights), trained by SGD as before. Of our 5,000 training stations, we train on 4,000 and hold out 1,000 for validation:



The training log loss keeps falling.

The validation log loss bottoms out at epoch 32, then climbs: **overfitting**.

Regularization

So far we tried **weight decay**, similar to ridge, minimizing:

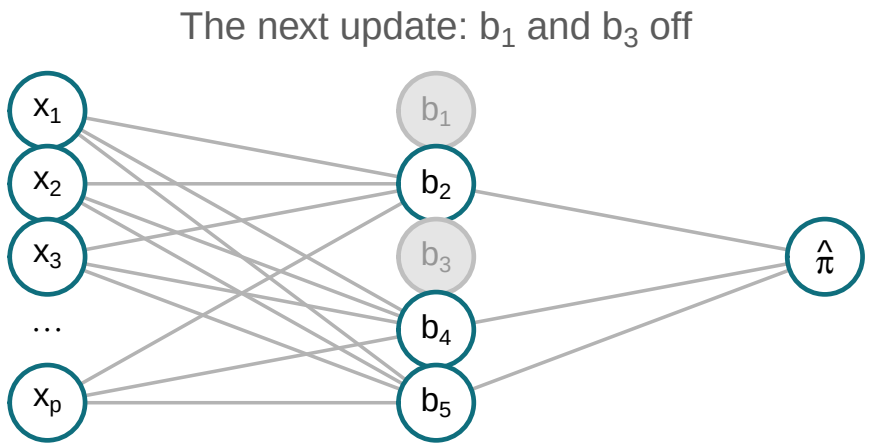
$$n \times \text{log loss} + \lambda \times (\text{sum of every squared weight})$$

Two more ways to keep a network from fitting noise:

1. **Early stopping**: stop updating before training converges, at the validation minimum. The weights stay nearer their small starting values. The number of epochs is a tuning parameter, like boosting's B.
2. **Dropout**: at each update, switch off a random share of the derived features.

All three are tuning parameters, chosen on held-out stations.

Dropout

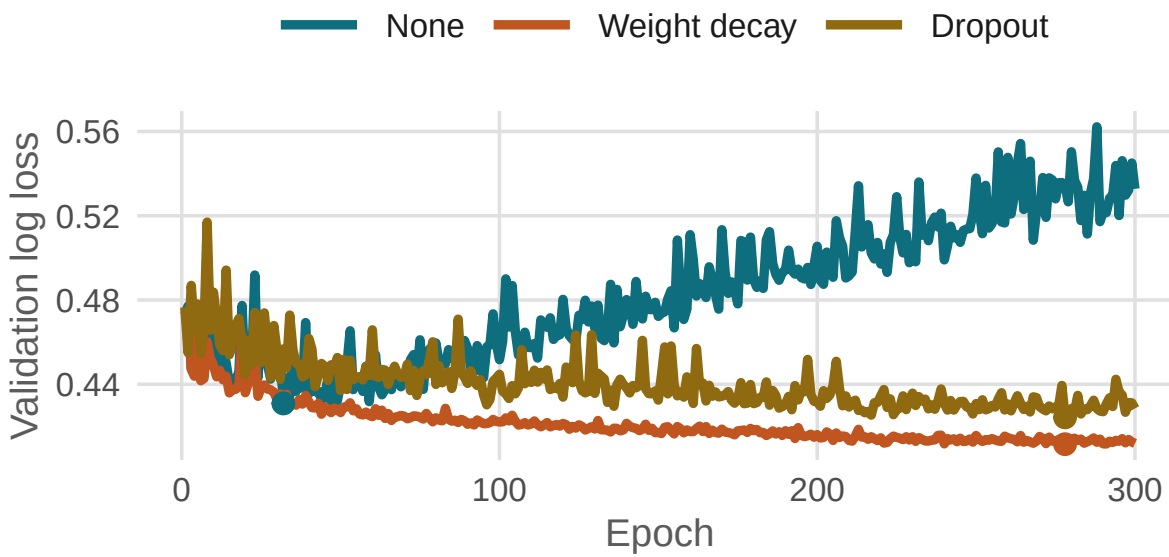


At each update, we switch off each derived feature with probability ϕ, the **dropout rate**. The survivors are scaled up by $1/(1-\phi)$, and only they are updated.

Why does this help? Any derived feature may be off at the next update, so none can rely on another to fix its errors. Each must be useful alone, which makes a few stations' noise harder to fit. The idea comes from random forests, where each split sees a random m of the p features.

To predict, we switch every derived feature back on.

Regularizing our network



Our wide network, on the validation stations. Dots: where early stopping would stop. With no regularization, the best is 0.431, at epoch 32. Then it climbs to 0.533.

With weight decay, the climb is gone: 0.412.

Dropout (ϕ = **0.5**) also stays flat: 0.424. Either way, we no longer have to stop at exactly the right epoch.

Roadmap

Training big networks

Overfitting

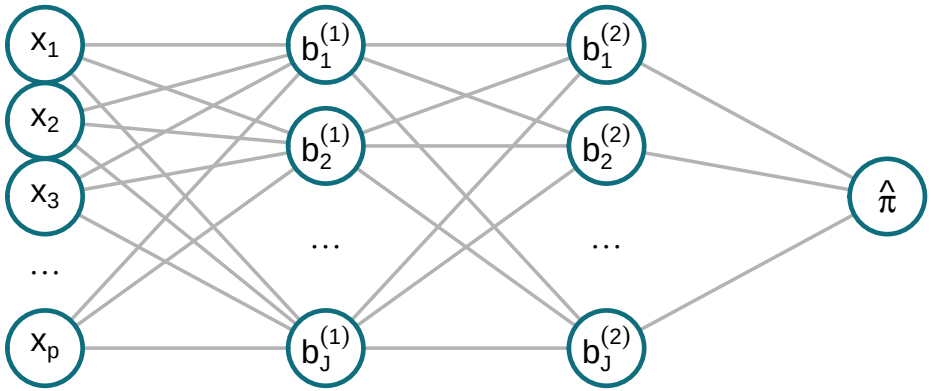
Going deep

Reading tally sheets

Deep networks

One hidden layer can already approximate any continuous $\pi(\boldsymbol{x})$, given enough derived features. But 'enough' can be enormous.

A **deep** network instead stacks hidden layers:



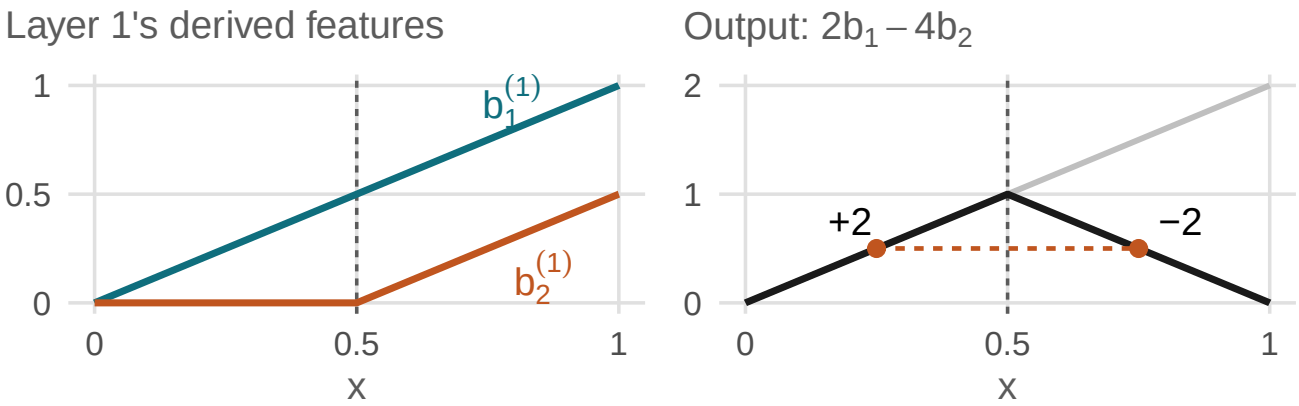
$$\boldsymbol{b}^{(1)} = g\left(\mathbf{w}_0^{(1)} + \boldsymbol{W}^{(1)}\boldsymbol{x}\right), \quad \boldsymbol{b}^{(2)} = g\left(\mathbf{w}_0^{(2)} + \boldsymbol{W}^{(2)}\boldsymbol{b}^{(1)}\right), \quad \text{logit}\,\hat{\pi} = \beta_0 + \beta^\top \boldsymbol{b}^{(2)}$$

Read: the second layer builds its derived features from the first's.

The model is trained the same as before, but there is now a second layer for the backward pass to go through.

Why go deep?

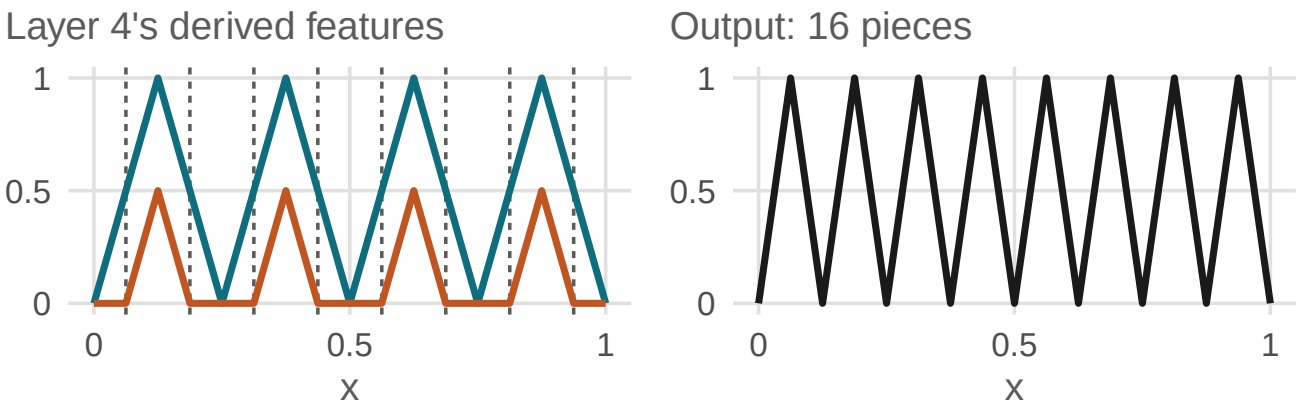
Last time, each ReLU derived feature added one knot. What does a second layer add? Let's build a network on one input, x between 0 and 1, choosing the weights by hand (no training).



The tent takes $[0, 1]$ back to $[0, 1]$, but **folded** in half: $x = 0.25$ and $x = 0.75$ both give 0.5.

Why go deep?

Layer 2 takes the tent as its input, $u = 2b_1^{(1)} - 4b_2^{(1)}$, and applies the same two ReLUs to it: $b_1^{(2)} = \max(0, u)$ and $b_2^{(2)} = \max(0, u - 0.5)$.



Each layer folds again, so each layer doubles the pieces. For 16 pieces, one layer needs 15 derived features; four layers use 8.

An additional layer adds a fold, which doubles the knots of every layer after it. So depth multiplies flexibility, while width only adds to it. That is why networks for words and pixels are deep.

Tuning a deep network

Every row is a **tuning parameter**:

Choice	`nnet`	SGD
Width J	10, by CV	256
Depth (hidden layers)	1	1
Weight decay λ	1, by CV	0.003 (PyTorch's scale)
Dropout rate ϕ	–	0.5
Step size α	– (quasi-Newton)	0.05
Minibatch n_b	– (all n)	32
Epochs	– (until converged)	early stopping

These parameters all interact, and CV over all of them can be prohibitively expensive. In practice we use a validation set (as we have done), start from sensible defaults, and try multiple random starts.

Roadmap

Training big networks

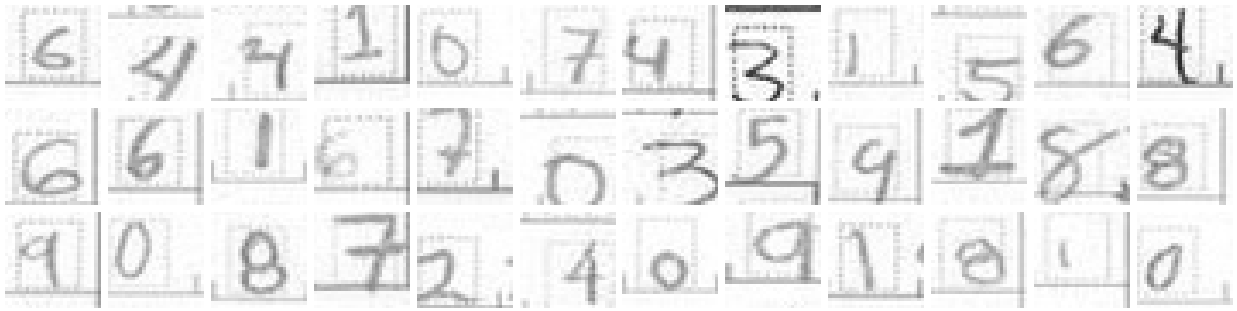
Overfitting

Going deep

Reading tally sheets

Reading tally sheets

Let's try a more complicated real-world example: polling station tally sheets from Mexico's 2015 election. Torres and Cantú (2022) clip every handwritten digit:



Each digit is $28 \times 28 = 784$ pixels. Each pixel is an input feature: how much ink, from 0 to 1. The outcome: which digit (0–9) are we looking at?

We have 21,844 training digits, 2,427 for validation, and 2,000 for testing.

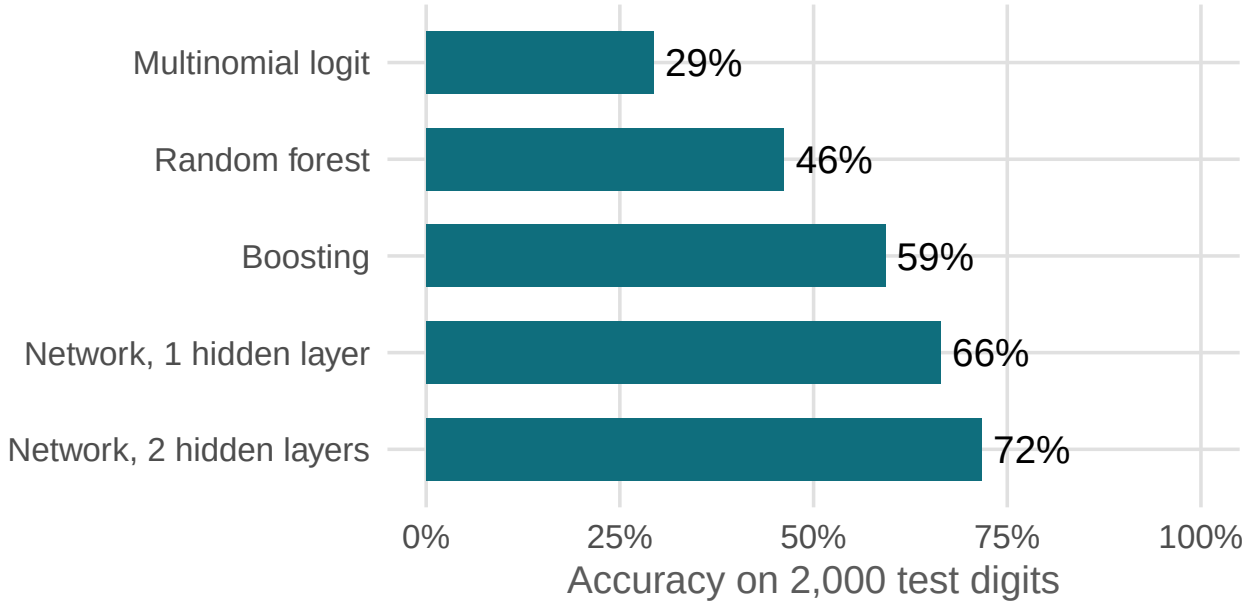
Which of our classifiers reads them best?

A network on pixels, in PyTorch

```
model = nn.Sequential(
    nn.Flatten(),                          # 28 x 28 -> 784 inputs
    nn.Linear(784, 256), nn.ReLU(),        # 256 derived features
    nn.Dropout(0.5),                       # dropout, phi = 0.5
    nn.Linear(256, 10))                    # 10 digits
loss_fn = nn.CrossEntropyLoss()            # log loss, multinomial
opt = torch.optim.SGD(model.parameters(), lr=0.05,
                      weight_decay=1e-4)
# X: training digits (n x 28 x 28 ink); y: their labels, 0-9
loader = DataLoader(TensorDataset(X, y), batch_size=32,
                    shuffle=True)          # shuffle, split into minibatches
for epoch in range(60):
    model.train()                          # dropout on
    for xb, yb in loader:                  # minibatches
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)      # forward pass
        loss.backward()                    # backward pass
        opt.step()                         # one update
    model.eval()                           # dropout off: validate,
                                           # keep the best epoch
```

The final output layer is a multinomial logit on the derived features.

Our classifiers on pixels



There are clear returns to flexibility. But even the multi-layer network tops out at 72%. More soon!

References

- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. – **10.2, 10.7**. statlearning.com.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016) *Deep Learning*. MIT Press – chs. 6–8. deeplearningbook.org.
- Torres, M. & Cantú, F. (2022) Learning to see: Convolutional neural networks for the analysis of social science data. *Political Analysis* 30(1), 113–131. Data: Harvard Dataverse, doi:10.7910/DVN/UNYOLF.
- Telgarsky, M. (2016) Benefits of depth in neural networks. *Conference on Learning Theory* (COLT), PMLR 49.